

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und strukturiertes Dokument wurde eine eigene Pythonlibrary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen 'Papulator' auf meinem GitHub zu finden. Diese soll stetig ausgearbeitet werden. Der Stand dieses Protokolls ist der 29.04.2026.

Inhaltsverzeichnis zur Auswertung

- [Definition der Versuchsvariablen](#)
 - [Import aller genutzen Libraries](#)
 - [Auswertung der Aufgaben](#)
 - [Aufgabe 1](#)
 - [Aufgabe 2](#)
 - [Aufgabe 3](#)
 - [Aufgabe 4](#)
 - [Aufgabe 5](#)
 - [Aufgabe 6](#)
-
-

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

```
In [1]: versuchsnummer = '241'
versuchsname = 'RLC-Glieder' # Auf Leerzeichen verzichten
aufgabe = '0z'
plt_save_folder = f'../img/plots/'
```

Import aller genutzen Libraries

```
In [2]: # Eigene Pythonlibrary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
```

```

import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re

from io import StringIO

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

```

/home/anonymous/Documents/GitHub/PAP-2/Versuche/241/Python/Papulator.py:104: SyntaxWarning: invalid escape sequence '\p'
 ...

Auswertung der Aufgaben

In [3]: *# Definition der Ungenauigkeiten*

```

V_step_err = 3
t_step_err = 5

def calc_step_err(voltage_div, time_div):
    voltage_step = voltage_div / 32
    time_step = time_div / 50

    return voltage_step * V_step_err, time_step * t_step_err

# Bauteile -> Prozentuale Fehler!
err_R_p = 0.05
err_C_p = 0.10

```

Aufgabe 1

In [4]: *# M1: 0.1 V/div, 50 us/div*
 v_s_err, t_s_err = calc_step_err(0.1, 50)
 M1_res = pap.round_sig_digs(t_s_err, 376)

print(f'Für M1 ergibt sich eine Halbwertszeit von: {M1_res[2]} us')

M2: 0.1 V/div, 5 us/div
 v_s_err, t_s_err = calc_step_err(0.1, 5)
 M2_res = pap.round_sig_digs(t_s_err, 31.2)

print(f'Für M2 ergibt sich eine Halbwertszeit von: {M2_res[2]} us')

M3: 0.1 V/div, 5 us/div
 v_s_err, t_s_err = calc_step_err(0.1, 5)
 M3_res = pap.round_sig_digs(t_s_err, 39.7)

print(f'Für M3 ergibt sich eine Halbwertszeit von: {M3_res[2]} us')

M4: 0.1 V/div, 20 us/div
 v_s_err, t_s_err = calc_step_err(0.1, 20)
 M4_res = pap.round_sig_digs(t_s_err, 30.2)

print(f'Für M4 ergibt sich eine Halbwertszeit von: {M4_res[2]} us')

Für M1 ergibt sich eine Halbwertszeit von: 376 μ s
 Für M2 ergibt sich eine Halbwertszeit von: 31.2 μ s
 Für M3 ergibt sich eine Halbwertszeit von: 39.7 μ s
 Für M4 ergibt sich eine Halbwertszeit von: 30.2 μ s

In [5]: *# Zeitkonstante experimentell*
 T_half = sp.symbols(r'T_{1/2}')

tau = T_half/sp.ln(2)

values = np.array([
 [M1_res[0], M1_res[1]],
 [M2_res[0], M2_res[1]],
 [M3_res[0], M3_res[1]],
 [M4_res[0], M4_res[1]]

```

])

temp_res = pap.do_it(tau, [T_half], values, [], print_formula=False)

res_in_pm = []
for i, n in enumerate(temp_res):
    str = pap.round_sig_digs(temp_res[i][1], temp_res[i][0])
    res_in_pm.append(str[2])

print(res_in_pm)

['542 \pm 7', '45.0 \pm 0.7', '57.3 \pm 0.7', '44 \pm 3']

```

```

In [6]: # Bauteile
R_vals = np.array([470.0, 4.7, 47.0, 47.0])
R_err = R_vals * err_R_p

C_vals = np.array([1,10,1,1])
C_err = C_vals * err_C_p

R_Bauteil = []
C_Bauteil = []
for i in range(len(R_vals)):
    temp = pap.round_sig_digs(R_err[i], R_vals[i])
    R_Bauteil.append(temp[2])
    temp = pap.round_sig_digs(C_err[i], C_vals[i])
    C_Bauteil.append(temp[2])

print(R_Bauteil)
print(C_Bauteil)

['470 \pm 24', '4.70 \pm 0.24', '47.0 \pm 2.4', '47.0 \pm 2.4']
['1.00 \pm 0.10', '10.0 \pm 1.0', '1.00 \pm 0.10', '1.00 \pm 0.10']

```

```

In [7]: # Zeitkonstante theoretisch
tau_2 = sym.R * sym.C

n=0
values = []
while n < len(R_vals):
    values.append(
        [R_vals[n], R_err[n],
         C_vals[n], C_err[n]]
    )
    n = n + 1

# values = np.array([
#     R_vals[n], R_err[n],
#     C_vals[n], C_err[n]
# ])

temp_res_1 = pap.do_it(tau_2, [sym.R, sym.C], values, [], print_formula=False)

res_in_pm_1 = []
for i, n in enumerate(temp_res_1):
    str = pap.round_sig_digs(temp_res_1[i][1], temp_res_1[i][0])
    res_in_pm_1.append(str[2])

abw_list = []
for i in range(4):
    abw = pap.std_abw(temp_res[i][0], temp_res_1[i][0], temp_res[i][1], temp_res_1[i][1])
    abw_list.append(abw)

print(abw_list)

[np.float64(1.37), np.float64(0.37), np.float64(1.94), np.float64(0.57)]

```

Aufgabe 2

```

In [ ]:

```

Aufgabe 3

```

In [8]: aufgabe = '3a'
filename = "A3/Amplitudenplot_Hochpass.txt"

# Read file
with open(filename, "r") as f:
    lines = f.readlines()

```

```

blocks = []
current_block = []

for line in lines:
    line = line.strip()

    if not line:
        continue

    # Detect new block
    if line.startswith("Hz"):
        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)

    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)

# Custom labels and colors
labels = ["Tiefpass", "Hochpass"]
colors = [c.LIGHT_GREEN, c.BLUE]

# Amplituden plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["dBV"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

# Setting lines
plt.hlines(-6.3, 10**2, 10**5, color=c.LIGHT_GREEN, linestyle=':')
plt.xlim(9.7**2, 10.1**5)

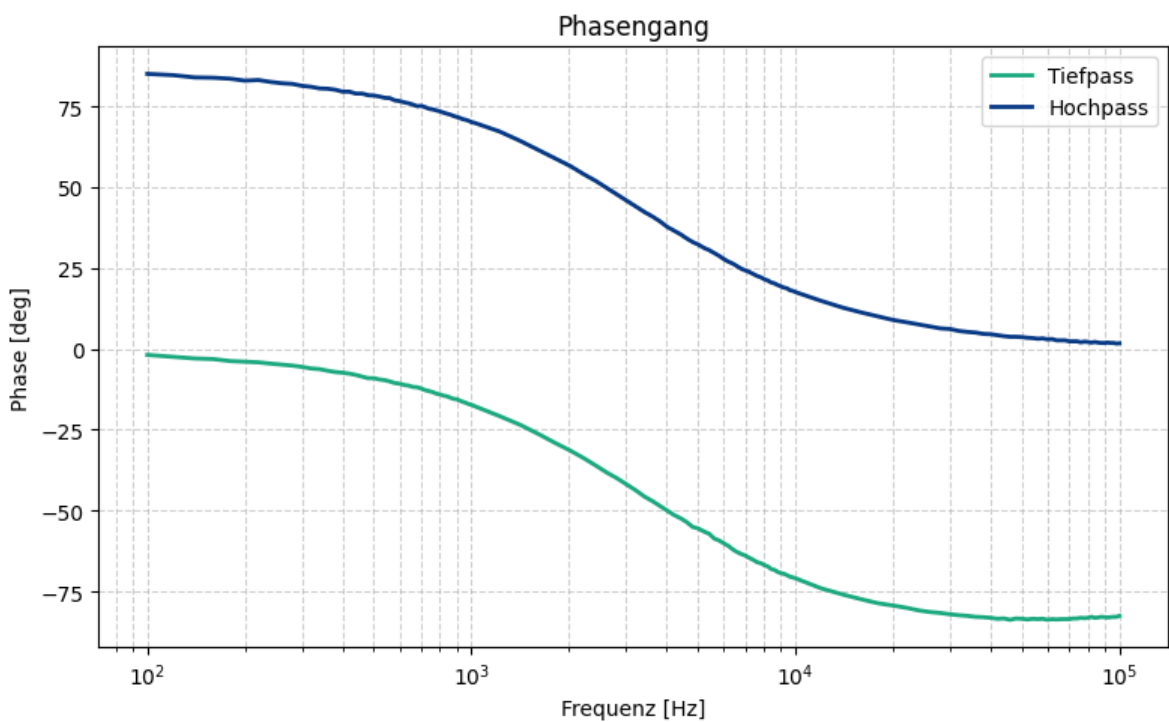
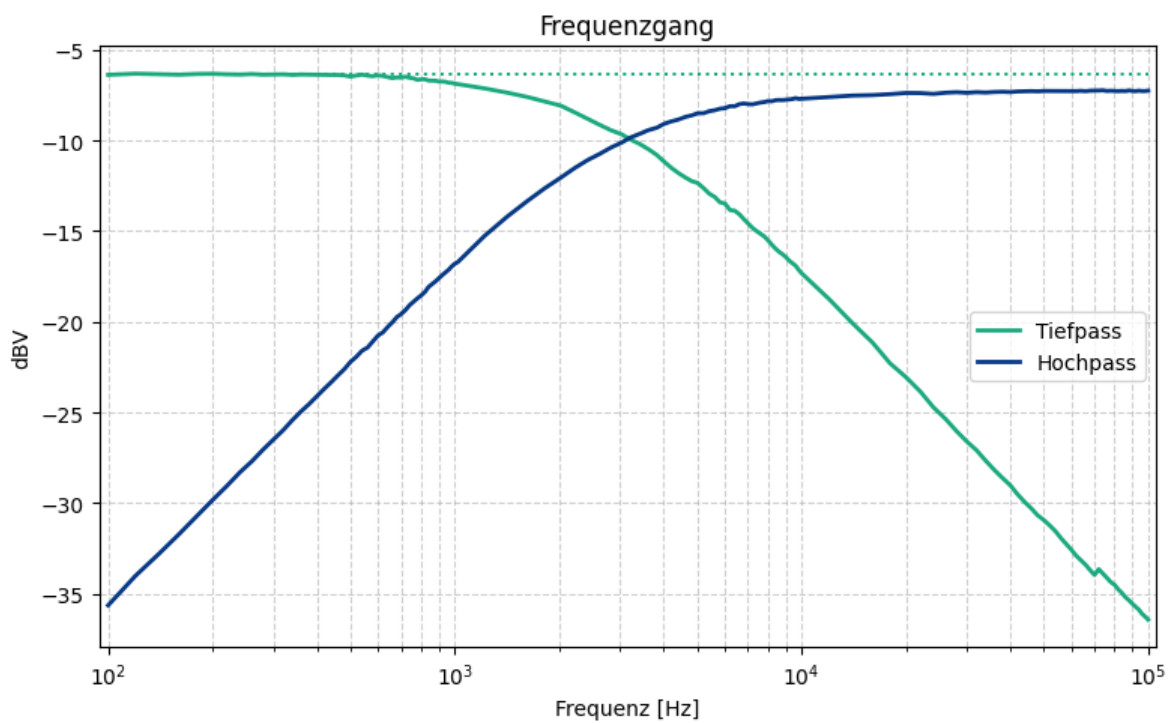
plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
plt.show()

# Phasen plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["deg"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

plt.xlabel("Frequenz [Hz]")
plt.ylabel("Phase [deg]")
plt.title("Phasengang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Phasengang_{aufgabe}.pdf')
plt.show()

```



```
In [9]: labels = ["Tiefpass", "Hochpass"]
        colors = [c.LIGHT_GREEN, c.BLUE]

# Amplituden plot
plt.figure(figsize=(8,5))

# Hilfsfunktion zur Berechnung/Darstellen der Asymptote
def plot_asymptote(ax, df, label, color, filter_type, start_freq=None, level_db=0):
    """
    Zeichnet die ideale Asymptote für Tiefpass oder Hochpass.
    filter_type: 'low' oder 'high'
    level_db: Der konstante Wert in dB
    """

    max_val = df['dBV'].max()
    threshold = max_val - 3.0

    if filter_type == 'low':
        idx_cutoff = df[df['dBV'] <= threshold].index.min()
        if pd.isna(idx_cutoff):
            idx_cutoff = len(df) - 1 # sicherheits fallback

        f_cutoff = df.loc[idx_cutoff, 'Hz']

    # Horizontale Linie bis zur Grenzfrequenz
    ax.hlines(level_db, df['Hz'].min(), f_cutoff, color=color, linestyle='--', alpha=0.7, linewidth=1.5)

    f_end = df['Hz'].max()
    delta_decades = np.log10(f_end / f_cutoff)
```

```

y_end = level_db - 20 * delta_decades

ax.plot([f_cutoff, f_end], [level_db, y_end], color=color, linestyle='--', alpha=0.7, linewidth=1.5)

elif filter_type == 'high':
    idx_cutoff = df[df['dBV'] <= threshold].index.max()
    if pd.isna(idx_cutoff):
        idx_cutoff = 0

    f_cutoff = df.loc[idx_cutoff, 'Hz']

    f_start = df['Hz'].min()
    delta_decades = np.log10(f_cutoff / f_start)
    y_start = level_db - 20 * delta_decades # Rückwärts rechnen

    ax.plot([f_start, f_cutoff], [y_start, level_db], color=color, linestyle='--', alpha=0.7, linewidth=1.5)

    # Horizontale Linie ab Grenzfrequenz
    ax.hlines(level_db, f_cutoff, df['Hz'].max(), color=color, linestyle='--', alpha=0.7, linewidth=1.5)

# Plot Loop
for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["dBV"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

    if i == 0:
        plot_asymptote(plt.gca(), df, labels[i], colors[i], 'low', level_db=-6.3)
    elif i == 1:
        plot_asymptote(plt.gca(), df, labels[i], colors[i], 'high', level_db=-7.3)

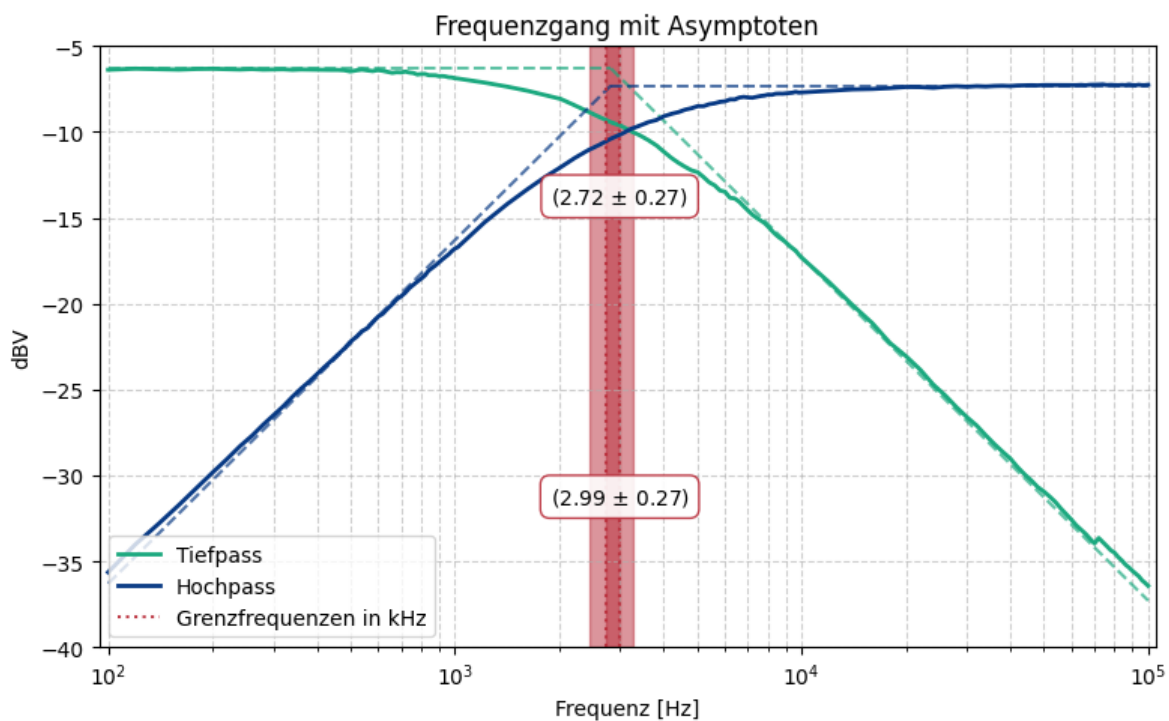
# plt.vlines(2.72 * 10**3, -37.5, -5, colors=c.PURPLE, label=fr'Tiefpass Grenzfrequenz (2.7 $\pm$ 0.27)')

pap.plot_errorlines(
    plt.gca(),
    [2.72 * 10**3, 2.99 * 10**3],
    0.27 * 10**3,
    -40,
    -5,
    add_label = True,
    label=fr'Grenzfrequenzen in kHz',
    names=[fr'(2.72 $\pm$ 0.27)', fr'(2.99 $\pm$ 0.27)']
)

plt.xlim(9.7**2, 10.1**5)
plt.ylim(-40, -5)

plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang mit Asymptoten")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}_mit_Asymptoten.pdf')
plt.show()

```



```
In [10]: # Einheiten in kHz
grenzfreq_tief = 2.72 * 10**3
grenzfreq_hoch = 2.99 * 10**3

err_grenzfreq = calc_step_err(0, 2.72 * 10**3)[1] # Falsche Div Size

print(pap.round_sig_digs(err_grenzfreq, grenzfreq_tief)[2])
print(pap.round_sig_digs(err_grenzfreq, grenzfreq_hoch)[2])

abw_exp = pap.std_abw(grenzfreq_hoch, grenzfreq_tief, err_grenzfreq, err_grenzfreq)

print(fr'Standarabweichung zwischen den Experimentellen Werten: {abw_exp}')
```

2700 \pm 300
3000 \pm 300
Standarabweichung zwischen den Experimentellen Werten: 0.7

```
In [11]: aufgabe = '3b'

freq_gen = np.arange(1, 11, 1) # 10^3 Hz

hochpass_phasenverschiebung = np.array([200, 80, 46, 25, 18, 13, 9.6, 7.6, 6, 4.6])
err_hochpass_phasenverschiebung = np.array([10, 10, 1, 1, 1, 0.1, 0.1, 0.1, 0.1, 0.1])

phi_hochpass = 360 * freq_gen * hochpass_phasenverschiebung / 1000
err_phi_hochpass = 360 * freq_gen * err_hochpass_phasenverschiebung / 1000

tiefpass_phasenverschiebung = np.array([60, 58.4, 51.6, 42.4, 34.8, 30.8, 28.8, 26.4, 22.8, 20])
err_tiefpass_phasenverschiebung = np.array([10, 10, 1, 1, 1, 0.1, 0.1, 0.1, 0.1, 0.1])

phi_tiefpass = 360 * freq_gen * tiefpass_phasenverschiebung / 1000
err_phi_tiefpass = 360 * freq_gen * err_tiefpass_phasenverschiebung / 1000

# Fit
def arctan(f, f_g):
    return np.arctan(f_g / f) * (360 / (2 * np.pi)) # f_g ist Grenzfrequenz

x_fit = np.arange(1, 10, 0.01)

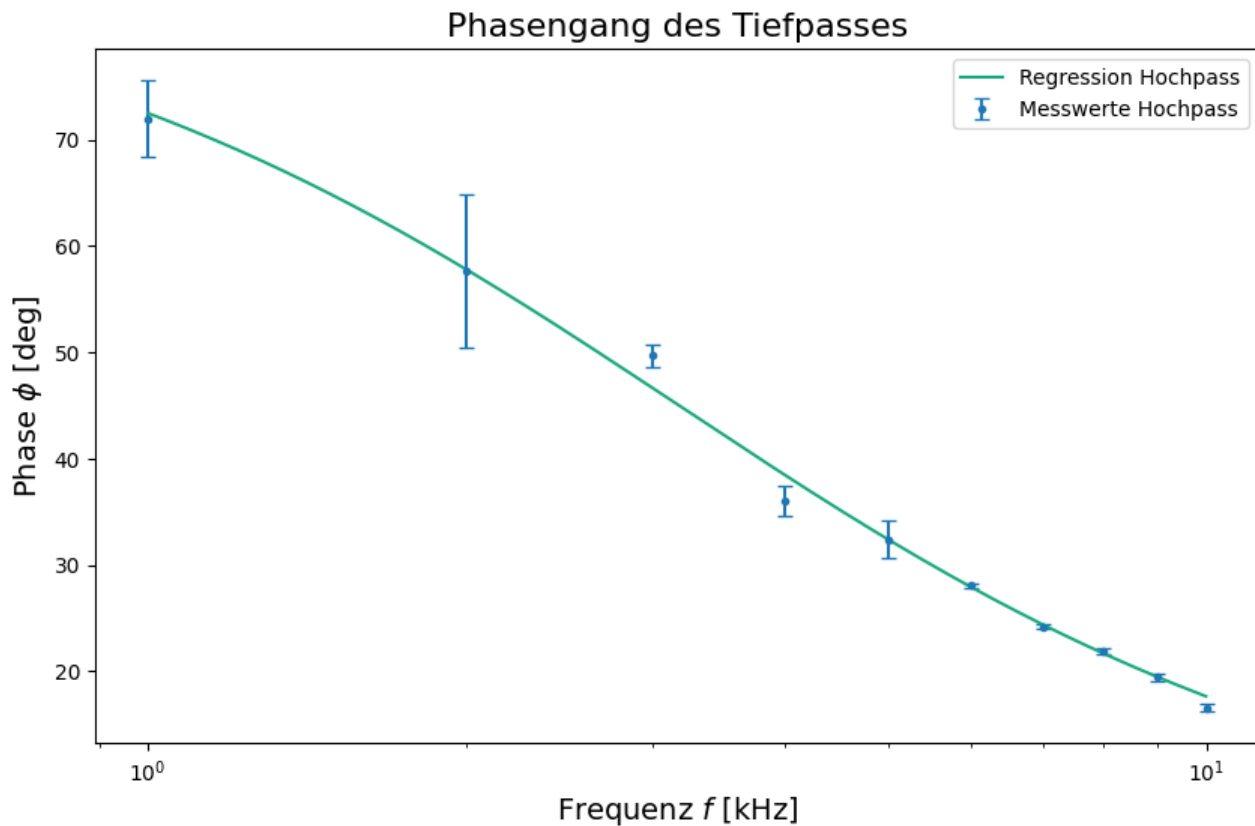
# Plot
plt.figure(figsize = (10, 6))
# plt.errorbar(freq_gen, phi_tiefpass, yerr = err_phi_tiefpass, fmt = '.', capsize = 3.5, label = 'Messwerte Tiefpass')
plt.errorbar(freq_gen, phi_hochpass, yerr = err_phi_hochpass, fmt = '.', capsize = 3.5, label = 'Messwerte Hochpass')

# popt, pcov = curve_fit(arctan, freq_gen, phi_tiefpass, sigma = err_phi_tiefpass)
# plt.plot(x_fit, arctan(x_fit, *popt), label = 'Regression Tiefpass')

popt, pcov = curve_fit(arctan, freq_gen, phi_hochpass, sigma = err_phi_hochpass)
plt.plot(x_fit, arctan(x_fit, *popt), label = 'Regression Hochpass', color=c.LIGHT_GREEN)

plt.title('Phasengang des Tiefpasses', size = 16)
plt.xlabel(fr'Frequenz $f$ [kHz]', size = 14)
plt.ylabel(fr'Phase $\phi$ [deg]', size = 14)
```

```
plt.xscale('log')
plt.legend(loc='best')
plt.savefig(fr'{plt_save_folder}Phasengnag_{aufgabe}.pdf')
plt.show()
```



Aufgabe 4

```
In [12]: aufgabe = '4a'
filename = "A4/Amplitudenplot_Alle.txt"

# Read file
with open(filename, "r") as f:
    lines = f.readlines()

blocks = []
current_block = []

for line in lines:
    line = line.strip()

    if not line:
        continue

    # Detect new block
    if line.startswith("Hz"):
        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)
    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)
```



```

# Custom labels and colors
labels = [fr"Widerstand  $R_1 = (1000 \pm 50) \Omega$ ", fr"Widerstand  $R_2 = (220 \pm 11) \Omega$ ", fr"Widerstand  $R_3 = (47.0 \pm 2.4) \Omega$ "]
colors = [c.LIGHT_GREEN, c.BLUE, c.YELLOW]

# Amplituden plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["dBV"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

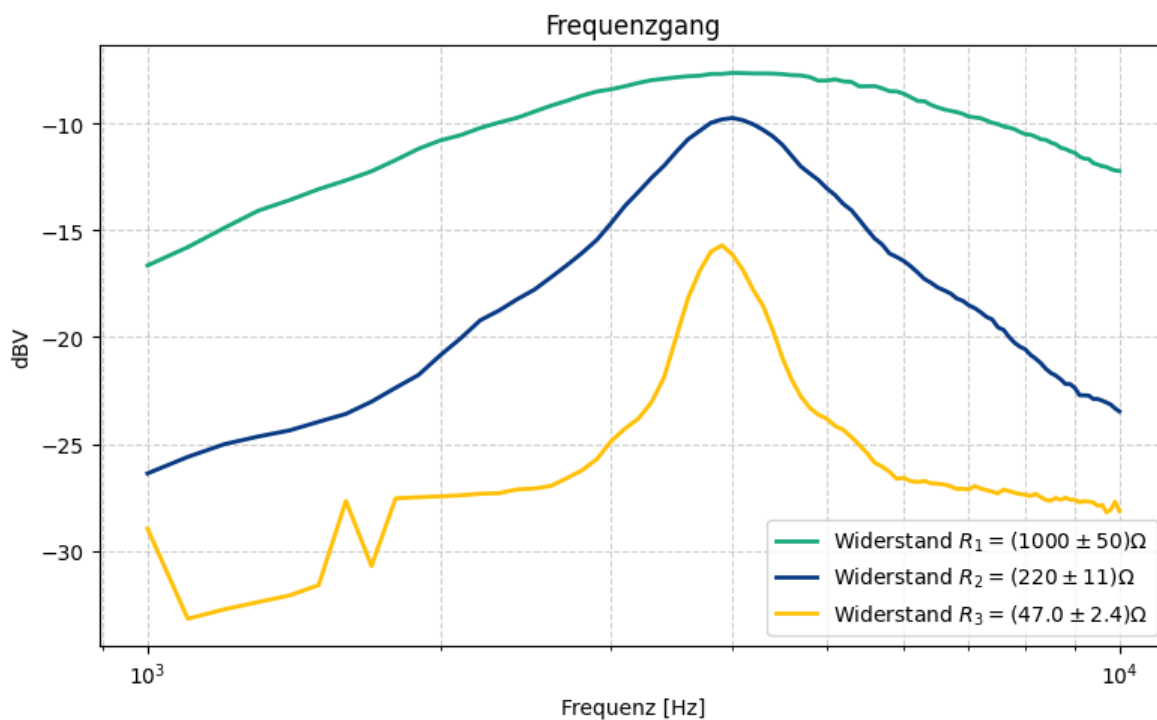
plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
plt.show()

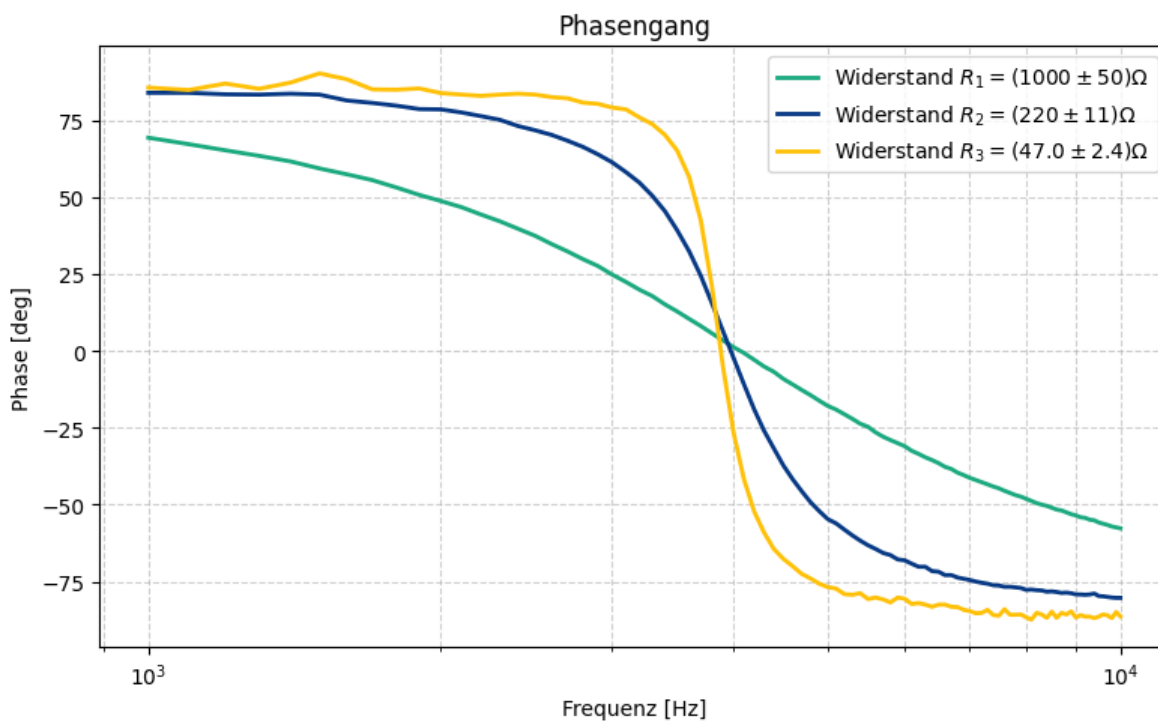
# Phasen plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["deg"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

plt.xlabel("Frequenz [Hz]")
plt.ylabel("Phase [deg]")
plt.title("Phasengang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Phasengang_{aufgabe}.pdf')
plt.show()

```





```
In [13]: def dB_to_Voltage(voltage):
        return 10 ** (voltage / 20)
```

```
In [14]: def find_maxima(dfs, labels):
        print(f'{\"Dataset\":<40} | {\"Frequency (Hz)\":<15} | {\"Max Amplitude (dBV)\":<20} | {\"Max Amplitude (V)\":<20}\"}')
        print("-" * 100)
        amplitude_list = []
        freq_list = []
        for i, df in enumerate(dfs):
            df_sorted = df.sort_values(by="Hz")

            # Find peaks in the dBV column
            peaks, _ = find_peaks(df_sorted["dBV"], distance=10, prominence=4)

            if len(peaks) == 0:
                # print(f'{labels[i]:<40} | No peaks found')
                continue

            for peak_idx in peaks:
                freq = df_sorted.iloc[peak_idx]["Hz"]
                amp = df_sorted.iloc[peak_idx]["dBV"]
                amplitude_list.append(amp)
                freq_list.append(freq)
                print(f'{labels[i]:<40} | {freq:<15.2f} | {amp:<20.2f} | {dB_to_Voltage(amp):<20.2f}')

        return amplitude_list, freq_list
```

```
In [15]: amps, freq = find_maxima(dfs, labels)
        amps = np.array([amps])

        Spannungen_Ausgang = dB_to_Voltage(amps)

        # print(fr'Die Ausgangsspannungen betragen: {np.round(Spannungen_Ausgang, 2)} Volt')
```

Dataset	Frequency (Hz)	Max Amplitude (dBV)	Max Amplitude (V)
Widerstand $R_1 = (1000 \pm 50) \Omega$	4000.00	-7.64	0.42
Widerstand $R_2 = (220 \pm 11) \Omega$	4000.00	-9.75	0.33
Widerstand $R_3 = (47.0 \pm 2.4) \Omega$	3900.00	-15.71	0.16

```
In [16]: aufgabe = '4b'
        filename = "A4/Amplitudenplot_Alle.txt"

        # Read file
        with open(filename, "r") as f:
            lines = f.readlines()

        blocks = []
        current_block = []

        for line in lines:
            line = line.strip()

            if not line:
                continue

            # Detect new block
            if line.startswith("Hz"):
```

```

        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)

    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)

# Custom labels and colors
labels = [fr"Widerstand  $R_1 = (1000 \pm 50) \Omega$ ", fr"Widerstand  $R_2 = (220 \pm 11) \Omega$ ", fr"Widerstand  $R_3 = (47.0 \pm 2.4) \Omega$ "]
colors = [c.LIGHT_GREEN, c.BLUE, c.YELLOW]

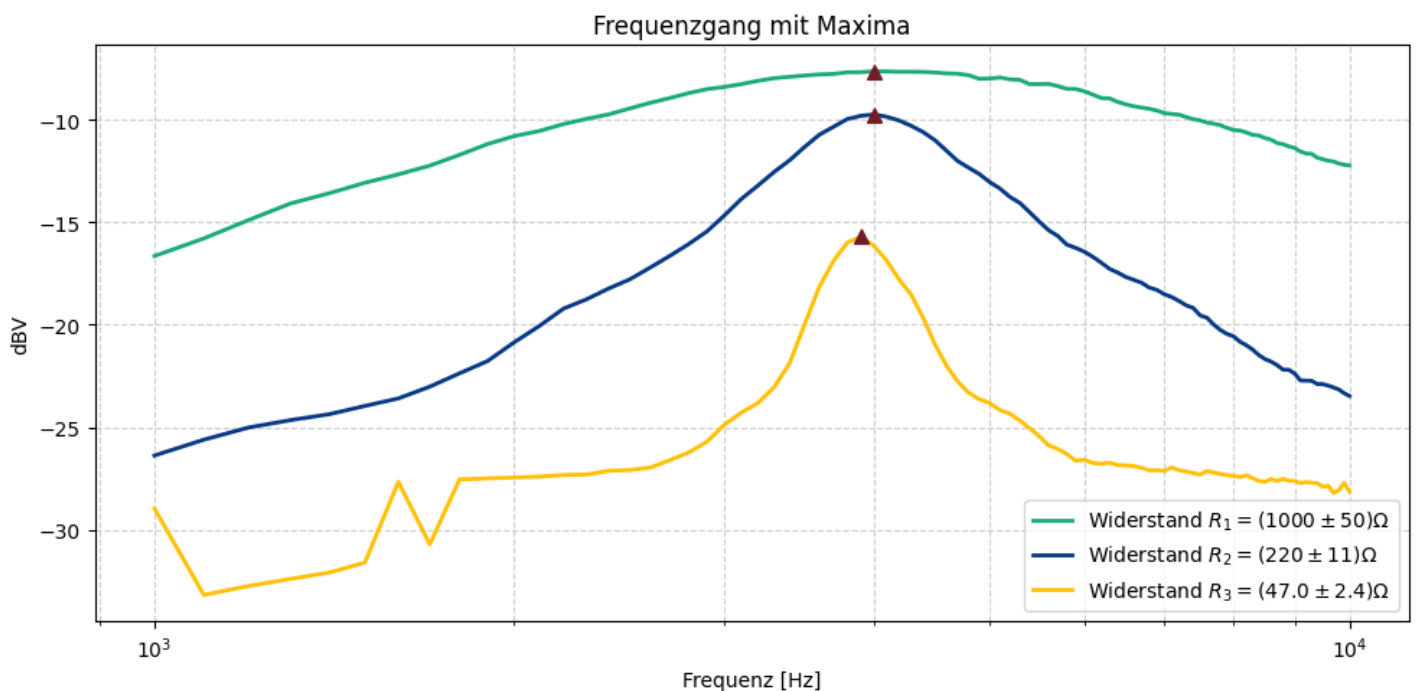
plt.figure(figsize=(10, 5))
for i, df in enumerate(dfs):
    df_sorted = df.sort_values(by="Hz")
    peaks, _ = find_peaks(df_sorted["dBV"], distance=5, prominence=4)

    plt.semilogx(df_sorted["Hz"], df_sorted["dBV"], label=labels[i], color=colors[i], linewidth=2)

# Plot markers for peaks
if len(peaks) > 0:
    plt.scatter(df_sorted.iloc[peaks]["Hz"],
                df_sorted.iloc[peaks]["dBV"],
                color=c.WINE_RED, s=50, zorder=5, marker='^',)

plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang mit Maxima")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}_mit_Maxima.pdf')
plt.show()

```



```

In [17]: # Import der Messwerte
Kapa = 47
Kapa_array = np.array([47, 47, 47]) * 1e-9 #nF ->F
err_Kapa_array = Kapa_array * err_C_p
widerstand = np.array([1, 0.22, 0.047]) * 1000 # kOhm -> Ohm

# linke Bandbreite in kHz

```

```

linksBB = np.array([2.01, 3.28, 3.56])

# Rechte Bandbreite in kHz
rechtsBB = np.array([8.43, 4.88, 4.31])

# Frequenz für das Maximum in kHz
maxFreq = np.array([4.15, 4.06, 3.91])
err_maxFreq = np.array([0.10, 0.10, 0.10])

deltaFreq1 = np.abs(maxFreq - linksBB)
deltaFreq2 = np.abs(maxFreq - rechtsBB)

BB = np.abs(deltaFreq1 + deltaFreq2)/2
BB = np.abs(linksBB - rechtsBB)

err_BB = BB * 0.05

induk = 1/((2 * sp.pi * sym.f)**2 * sym.C)

params = [sym.f, sym.C]

values = np.column_stack([
    maxFreq* 1e3, maxFreq * 0.05e3,
    Kapa_array, err_Kapa_array,
])

temp_induk = pap.do_it(induk, params, values, [], print_formula=False)
print(temp_induk)

```

```

[[0.03129293 0.00442549]
 [0.03269568 0.00462387]
 [0.03525242 0.00498544]]

```

In [18]: # Erstellen der Latex Tabelle via Pandas

```

# Strings erstellen
induk_L_1 = []
widerstand_str = []
ResoFreq_str = []
BB_str = []
for i in range(len(widerstand)):
    widerstand_str.append(pap.round_sig_digs(widerstand[i] * err_R_p, widerstand[i])[2])
    ResoFreq_str.append(pap.round_sig_digs(err_maxFreq[i], maxFreq[i])[2])
    BB_str.append(pap.round_sig_digs(err_BB[i], BB[i])[2])
    induk_L_1.append(pap.round_sig_digs(temp_induk[i][1] * 10**2, temp_induk[i][0] * 10**2)[2])

df = pd.DataFrame({
    fr'\text{{Widerstand}} [\Ohm]': widerstand_str,
    fr'\text{{Resonanzfrequenz}} [kHz]': ResoFreq_str,
    fr'\text{{Bandbreite}} [kHz]': BB_str,
    fr'\text{{Induktivität}} L_1 [H]': induk_L_1
})

# Generate LaTeX table
latex_table = df.to_latex(
    index=False,
    caption=fr'Messwerte und berechnete Induktivität mit einer Induktivität von ({Kapa} $\pm$ {Kapa * err_C_p}) nF',
    label='tab:messwerte',
    column_format='C|C|C|C',
    escape=False
)

print(latex_table)

```

```

\begin{table}
\caption{Messwerte und berechnete Induktivität mit einer Induktivität von (47 $\pm$ 4.7) nF}
\label{tab:messwerte}
\begin{tabular}{C|C|C|C}
\toprule
\text{Widerstand} [\Ohm] & \text{Resonanzfrequenz} [kHz] & \text{Bandbreite} [kHz] & \text{Induktivität} L_1 [H] \\
\midrule
1000 $\pm$ 50 & 4.15 $\pm$ 0.10 & 6.4 $\pm$ 0.3 & 3.1 $\pm$ 0.4 \\
220 $\pm$ 11 & 4.06 $\pm$ 0.10 & 1.60 $\pm$ 0.08 & 3.3 $\pm$ 0.5 \\
47.0 $\pm$ 2.4 & 3.91 $\pm$ 0.10 & 0.75 $\pm$ 0.04 & 3.5 $\pm$ 0.5 \\
\bottomrule
\end{tabular}
\end{table}

```

In [19]: # Durchschnittliche Induktivität L_1

```

asStack = np.column_stack(temp_induk)

avg_induk_L_1 = np.mean(asStack[0])
err_avg_induk_L_1 = np.mean(asStack[1])

res = pap.round_sig_digs(err_avg_induk_L_1* 10**2, avg_induk_L_1 * 10**2)

```

```
print(fr'Die durchschnittliche Induktivität liegt bei: ({res[2]}) * 10**-2 H')
```

Die durchschnittliche Induktivität liegt bei: (3.3 \pm 0.5) * 10**-2 H

Gesamtwiderstand und Verluste

```
In [20]: # Bestimmung des Gesamtwiderstandes
R_ges = 2*sp.pi * sym.f * sym.L

induk_val = []
induk_err = []
for i in range(len(BB)):
    induk_val.append(temp_induk[i][0])
    induk_err.append(temp_induk[i][1])

values = np.column_stack([
    BB, err_BB,
    induk_val, induk_err
])

temp = pap.do_it(R_ges, [sym.f, sym.L], values, [], print_formula=False)
val_R_ges = temp[:,0] * 1000 # Ohm
err_R_ges = temp[:,1] * 1000 # Ohm

print(val_R_ges)

[1262.29571614  328.69279865  166.12309827]
```

```
In [21]: # Berechnung des Verlustwiderstandes
R_G_sym = sp.symbols(fr'R_\text{{ges}}')
R_ver = R_G_sym - sym.R

values = np.column_stack([
    val_R_ges, err_R_ges,
    widerstand, widerstand * err_R_p
])

temp = pap.do_it(R_ver, [R_G_sym, sym.R], values, [], print_formula=False)
val_R_ver = temp[:,0] # Ohm
err_R_ver = temp[:,1] # Ohm
```

```
In [22]: # Berechnung des Verlustwiderstandes über die Spannungen

U_E_sym, U_A_sym = sp.symbols(fr'U_\text{{E}}', fr'U_\text{{A}}')

R_ver_2 = sym.R * ((U_E_sym / U_A_sym) - 1)

params = [sym.R, U_E_sym, U_A_sym]

temp = 1
U_eing = np.array([temp, temp, temp])
err_U_eing = U_eing * 0.075

U_ausg = Spannungen_Ausgang[0]
temp = 0.05
err_U_ausg = np.array([temp, temp, temp])

values = np.column_stack([
    widerstand, widerstand * err_R_p,
    U_eing, err_U_eing,
    U_ausg, err_U_ausg
])

# Mit fehlerbehafteten Spannungen
temp = pap.do_it(R_ver_2, params, values, [], print_formula=False)
val_R_ver_2 = temp[:,0]
err_R_ver_2 = temp[:,1]

print('Verlustwiderstan:')
for i in range(3):
    print(pap.round_sig_digs(err_R_ver_2[i], val_R_ver_2[i])[2])

pap.std_abw(val_R_ver_2, val_R_ver, err_R_ver_2, err_R_ver)
```

Verlustwiderstan:
1400 \pm 300
460 \pm 120
240 \pm 90

```
Out[22]: array([2.87, 2.71, 1.28])
```

```
In [23]: # Erstellen der Latex-Tabelle

# Strings erstellen
wid_sum = []
wid_ver = []
```

```

for i in range(len(widerstand)):
    wid_sum.append(pap.round_sig_digs(err_R_ges[i], val_R_ges[i])[2])
    wid_ver.append(pap.round_sig_digs(err_R_ver[i], val_R_ver[i])[2])

df = pd.DataFrame({
    fr'\text{{Bandbreite}} f \text{{band}} \ [10^{{3}}\mathrm{{Hz}}]': BB_str,
    fr'\text{{Induktivität}} L_1 \ [10^{{-2}}\mathrm{{H}}]': induk_L_1,
    fr'\text{{Widerstand}} [\mathrm{{\Omega}}]': widerstand_str,
    fr'R + R_\text{{V}} [\mathrm{{\Omega}}]': wid_sum,
    fr'R_\text{{V}} [\mathrm{{\Omega}}]': wid_ver,
})

# Generate LaTeX table
latex_table = df.to_latex(
    index=False,
    caption=fr'Berechnung des Verlustwiderstands durch Bandbreite und Induktivität.',
    label='tab:messwerte_widerstande',
    column_format='C | C | C | C | C',
    escape=False
)

print(latex_table)

```

```

\begin{table}
\caption{Berechnung des Verlustwiderstands durch Bandbreite und Induktivität.}
\label{tab:messwerte_widerstande}
\begin{tabular}{C | C | C | C | C}
\toprule
\text{Bandbreite} f \text{band} \ [10^{{3}}\mathrm{Hz}] & \text{Induktivität} L_1 \ [10^{{-2}}\mathrm{H}] & \text{Widerstan}
d \ [\mathrm{\Omega}] & R + R_\text{V} \ [\mathrm{\Omega}] & R_\text{V} \ [\mathrm{\Omega}] \\
\midrule
6.4 \pm 0.3 & 3.1 \pm 0.4 & 1000 \pm 50 & 1260 \pm 190 & 260 \pm 200 \\
1.60 \pm 0.08 & 3.3 \pm 0.5 & 220 \pm 11 & 330 \pm 50 & 110 \pm 50 \\
0.75 \pm 0.04 & 3.5 \pm 0.5 & 47.0 \pm 2.4 & 166 \pm 25 & 120 \pm 30 \\
\bottomrule
\end{tabular}
\end{table}

```

Aufgabe 5

In [24]:

```

aufgabe = '5a'
filename = "A5/ResonanzkurvenAlleBauteile.txt"

```

```

# Read file
with open(filename, "r") as f:
    lines = f.readlines()

blocks = []
current_block = []

for line in lines:
    line = line.strip()

    if not line:
        continue

    # Detect new block
    if line.startswith("Hz"):
        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)

    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)

# Custom labels and colors

```

```

labels = [ "Kondensator", "Spule", "Widerstand"]
colors = [c.LIGHT_GREEN, c.BLUE, c.YELLOW]

# Amplituden plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["dBV"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

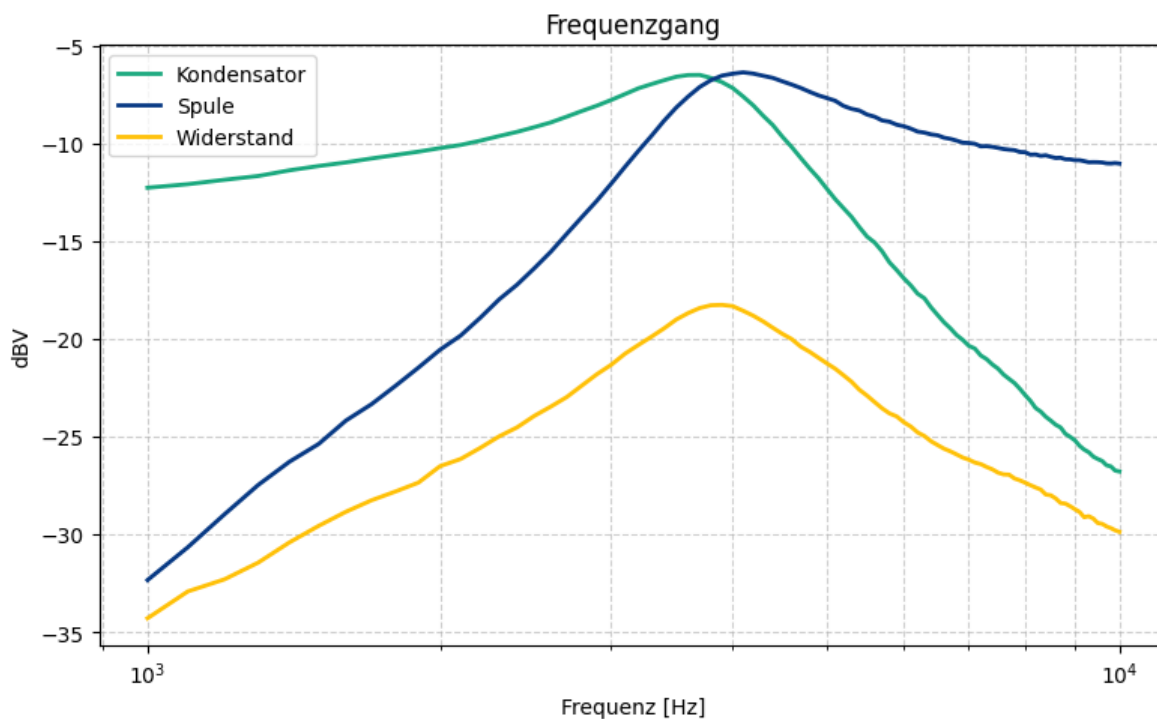
plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
plt.show()

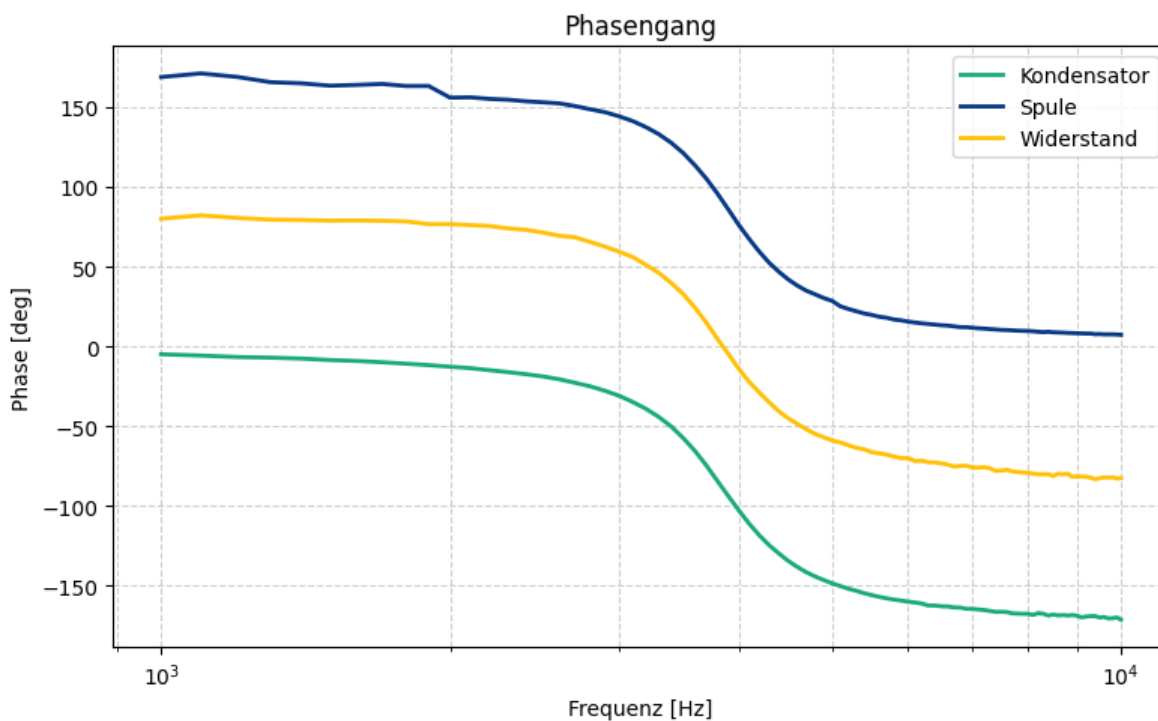
# Phasen plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["deg"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

plt.xlabel("Frequenz [Hz]")
plt.ylabel("Phase [deg]")
plt.title("Phasengang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Phasengang_{aufgabe}.pdf')
plt.show()

```





In [25]: `# Bestimmung der Induktivitäten je nach Bauteil`

```
resoFreq_R = 3.63
resoFreq_C = 3.91
resoFreq_L = 4.09

array_bauteile_freq = np.array([resoFreq_R, resoFreq_C, resoFreq_L])

Kapa_array = np.array([47, 47, 47]) * 1e-9 #nF ->F
err_Kapa_array = Kapa_array * err_C_p

induk = 1/((2 * sp.pi * sym.f)**2 * sym.C)

params = [sym.f, sym.C]

values = np.column_stack([
    array_bauteile_freq * 1e3, array_bauteile_freq * 0.05e3,
    Kapa_array, err_Kapa_array,
])

temp_induk = pap.do_it(induk, params, values, [], print_formula=False)
val_induk_R = temp_induk[0][0]
err_induk_R = temp_induk[0][1]

val_induk_C = temp_induk[1][0]
err_induk_C = temp_induk[1][1]

val_induk_L = temp_induk[2][0]
err_induk_L = temp_induk[2][1]

print(fr'Induktivität über den Widerstand: {pap.round_sig_digs(err_induk_R, val_induk_R)[2]}')
print(fr'Induktivität über den Kondensator: {pap.round_sig_digs(err_induk_C, val_induk_C)[2]}')
print(fr'Induktivität über die Spule: {pap.round_sig_digs(err_induk_L, val_induk_L)[2]}')

Induktivität über den Widerstand: 0.041 \pm 0.006
Induktivität über den Kondensator: 0.035 \pm 0.005
Induktivität über die Spule: 0.032 \pm 0.005
```

In [26]: `pap.std_abw(avg_induk_L_1, val_induk_L, err_avg_induk_L_1, err_induk_L)`

Out[26]: `np.float64(0.13)`

Aufgabe 6

```
In [27]: voltage = np.array([4.63, 2.84, 1.69, 1.03, 0.69, 0.47, 0.31]) # Amplituden
period_t = np.array([0.26, 0.27, 0.26, 0.26, 0.25, 0.26]) * 1e-3 # ms in s

calc_step_err(1, 0.2)
```

Out[27]: `(0.09375, 0.02)`

In [28]: `error_abs = 0.09375`


```
log_decrements = np.log(voltage[:-1] / voltage[1:])

err_n = error_abs / voltage[:-1]
err_np1 = error_abs / voltage[1:]

err_lambda = np.sqrt(err_n**2 + err_np1**2)

avg_log_decrements = np.mean(log_decrements)
err_avg_log_decrements = np.mean(err_lambda)

print(fr'Log Dek avg: {pap.round_sig_digs(err_avg_log_decrements, avg_log_decrements)[2]}')
```

Log Dek avg: 0.45 \pm 0.16

Bestimmung Dämpfungskonstante

```
In [29]: schwingDau_T = np.mean(period_t)
abs_err_t = 0.02 * 1e-3 # ms in s
err_schwingDau_T = np.sqrt(np.std(period_t)**2 + abs_err_t**2)
print(fr'Die Schwingdauer beträgt ({pap.round_sig_digs(err_schwingDau_T, schwingDau_T)[2]} s')

DampfKonst = sym.Lambda / sym.T

values = np.array([
    avg_log_decrements, err_avg_log_decrements,
    schwingDau_T, err_schwingDau_T
])

temp = pap.do_it(DampfKonst, [sym.Lambda, sym.T], values, [], print_formula=False)
val_dampf_konst = temp[0][0]
err_dampf_konst = temp[0][1]

print(fr'Die Dämpfungskonstante lautet {pap.round_sig_digs(err_dampf_konst, val_dampf_konst)[2]}')
```

Die Schwingdauer beträgt (0.000260 \pm 0.000021) s
Die Dämpfungskonstante lautet 1700 \pm 600

Widerstandsbestimmung

```
In [30]: avg_L1 = (avg_induk_L_1 + val_induk_L)/2
err_avg_L1 = np.sqrt(err_induk_L**2 + err_avg_induk_L**2)
print(fr'Die Induktivität der Spule L_1 beträgt ({pap.round_sig_digs(err_avg_L1, avg_L1)[2]} s')

ges_wid_A6 = 2 * sym.L * sym.delta

values = np.array([
    avg_L1, err_avg_L1,
    val_dampf_konst, err_dampf_konst
])

temp = pap.do_it(ges_wid_A6, [sym.L, sym.delta], values, print_formula=False)
val_ges_wid_A6 = temp[0][0]
err_ges_wid_A6 = temp[0][1]
print(fr'Der Gesamt widerstand beträgt: ({pap.round_sig_digs(err_ges_wid_A6, val_ges_wid_A6)[2]} Ohm')
```

Die Induktivität der Spule L_1 beträgt (0.033 \pm 0.007)
Der Gesamt widerstand beträgt: (110 \pm 50) Ohm

```
In [31]: # Verlustwiderstand
val_ver_wid_A6 = val_ges_wid_A6 - 47.0
err_ver_wid_A6 = np.sqrt(err_ges_wid_A6**2 + 2.4**2)

print(fr'Der Verlustwiderstand beträgt ({pap.round_sig_digs(err_ver_wid_A6, val_ver_wid_A6)[2]} Ohm)')
```

Der Verlustwiderstand beträgt (70 \pm 50 Ohm)

```
In [32]: #Abweichungen zu vorherigen Ergebnissen
print('VERLUST WIDERSTAND')
print(fr'Abweichung zu den Werten der Bandbreite: {pap.std_abw(val_ver_wid_A6, val_R_ver[2], err_ver_wid_A6, err_R_ver[2])}')
print(fr'Abweichung zu den Werten der Resonanzüberhöhung: {pap.std_abw(val_ver_wid_A6, val_R_ver_2[2], err_ver_wid_A6, err_R_ver_2[2])}')

print('=' * 10)

print('GESAMT WIDERSTAND')

ges_wid_A2 = val_R_ver_2[2] + 47
err_ges_wid_A2 = np.sqrt(err_R_ver_2[2]**2 + 2.4**2)

print(err_ges_wid_A2)

print(fr'Abweichung zu den Werten der Bandbreite: {pap.std_abw(val_ver_wid_A6, val_R_ges[2], err_ver_wid_A6, err_R_ges[2])}')
print(fr'Abweichung zu den Werten der Resonanzüberhöhung: {pap.std_abw(val_ver_wid_A6, ges_wid_A2, err_ver_wid_A6, err_ges_wid_A2)')')
```

VERLUST WIDERSTAND
Abweichung zu den Werten der Bandbreite: 0.98
Abweichung zu den Werten der Resonanzüberhöhung: 1.69
=====

GESAMT WIDERSTAND
90.84200543174374
Abweichung zu den Werten der Bandbreite: 1.86
Abweichung zu den Werten der Resonanzüberhöhung: 2.15

Reso Freq

```
In [33]: reso_freq_A6 = 1 / sym.T

values = np.array([
    schwingDau_T, err_schwingDau_T
])

temp = pap.do_it(reso_freq_A6, [sym.T], values, print_formula=False)
val_reso_freq_A6 = temp[0][0]
err_reso_freq_A6 = temp[0][1]
print(fr'Reso Freq: {pap.round_sig_digs(err_reso_freq_A6, val_reso_freq_A6)[2]}')
```

Reso Freq: 3800 \pm 300

```
In [34]: print(fr'StdAbw: {pap.std_abw(val_reso_freq_A6, 3910, err_reso_freq_A6, 100)}')
```

StdAbw: 0.2

Aufgabe 7

```
In [35]: aufgabe = '7a'
filename = "A7/Frequenzgang_mit_Resonanzfrequenz.txt"

# Read file
with open(filename, "r") as f:
    lines = f.readlines()

blocks = []
current_block = []

for line in lines:
    line = line.strip()

    if not line:
        continue

    # Detect new block
    if line.startswith("Hz"):
        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)

    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)

# Custom labels and colors
labels = ["Widerstand"]
colors = [c.LIGHT_GREEN]

# Amplituden plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["dBV"],
```

```

        label=labels[i],
        color=colors[i],
        linewidth=2
    )

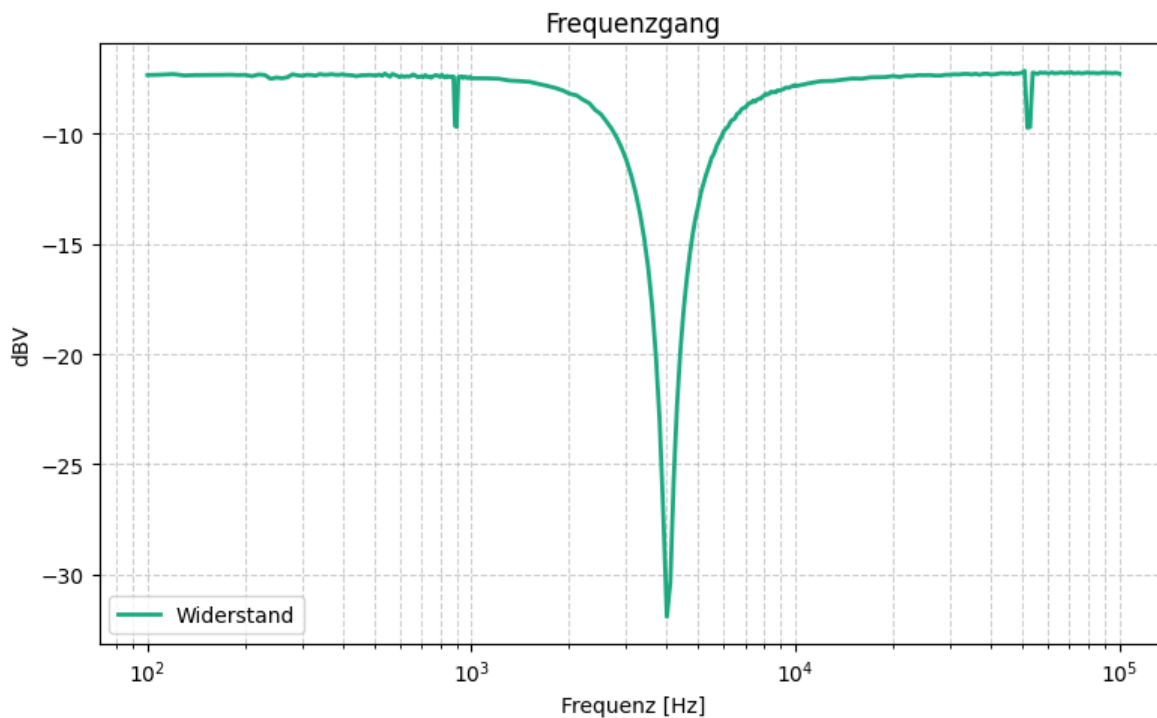
plt.xlabel("Frequenz [Hz]")
plt.ylabel("dBV")
plt.title("Frequenzgang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')
plt.show()

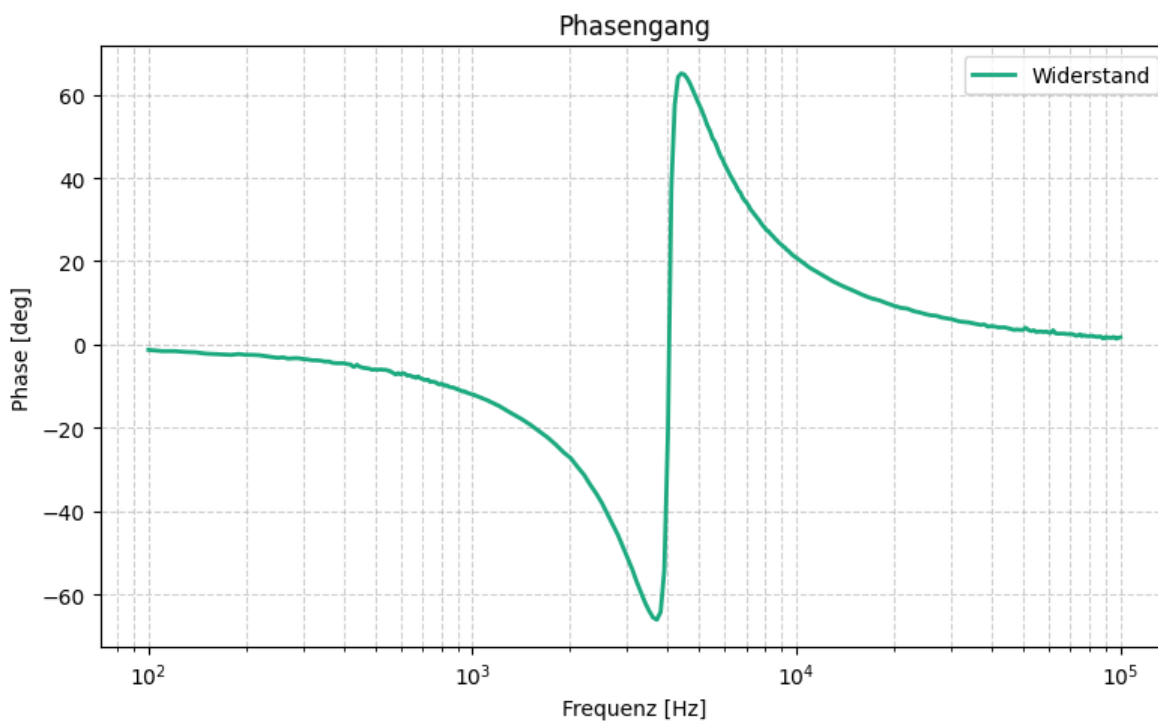
# Phasen plot
plt.figure(figsize=(8,5))

for i, df in enumerate(dfs):
    plt.semilogx(
        df["Hz"],
        df["deg"],
        label=labels[i],
        color=colors[i],
        linewidth=2
    )

plt.xlabel("Frequenz [Hz]")
plt.ylabel("Phase [deg]")
plt.title("Phasengang")
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(fr'{plt_save_folder}Phasengang_{aufgabe}.pdf')
plt.show()

```





```
In [36]: labels = ["Widerstand"]
colors = [c.LIGHT_GREEN]

fig, ax1 = plt.subplots(figsize=(10, 5))

for i, df in enumerate(dfs):
    ax1.semilogx(
        df["Hz"],
        df["dBV"],
        label=f"{labels[i]} Amplitude",
        color=colors[i],
        linewidth=2
    )

ax1.set_xlabel("Frequenz [Hz]")
ax1.set_ylabel("Amplitude [dBV]", color=colors[0])
ax1.tick_params(axis='y', labelcolor=colors[0])
ax1.grid(True, which="both", linestyle="--", alpha=0.6)

ax2 = ax1.twinx()

for i, df in enumerate(dfs):
    ax2.semilogx(
        df["Hz"],
        df["deg"],
        label=f"{labels[i]} Phase",
        color=c.BLUE,
        linewidth=2,
        # linestyle='--'
    )

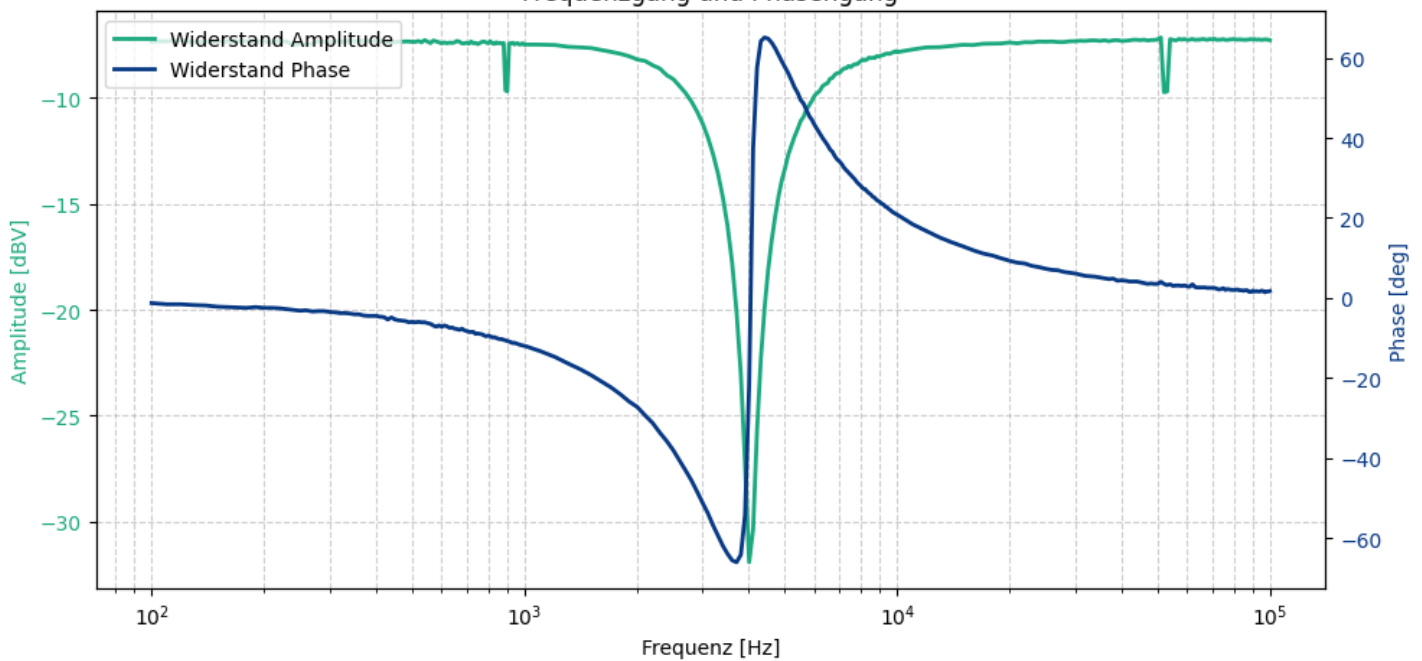
ax2.set_ylabel("Phase [deg]", color=c.BLUE)
ax2.tick_params(axis='y', labelcolor=c.BLUE)

lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left')

plt.title("Frequenzgang und Phasengang")
plt.tight_layout()

plt.savefig(f'{plt_save_folder}Frequenzgang_Phasengang_Kombiniert_{aufgabe}.pdf')
plt.show()
```

Frequenzgang und Phasengang



```
In [37]: f_teso = 1 / (2 * sp.pi * sp.sqrt(sym.L * sym.C))

values = np.array([
    avg_L1, err_avg_L1, #H
    47 * 1e-9, 2.4 * 1e-9 #nF -> F
])

print(values)

temp = pap.do_it(f_teso, [sym.L, sym.C], values, print_formula=False)
val_f_teso = temp[0][0]
err_f_teso = temp[0][1]
print(fr'Theoretische Resonanzfrequenz: {pap.round_sig_digs(err_f_teso, val_f_teso)[2]}')

[3.26490670e-02 6.53038306e-03 4.70000000e-08 2.40000000e-09]
Theoretische Resonanzfrequenz: 4100 \pm 400
```

```
In [38]: #Standard abweichungen

print(pap.std_abw(val_f_teso, 3800, err_f_teso, 300))

print(pap.std_abw(val_f_teso, 3910, err_f_teso, 100))

0.51
0.35
```

Aufgabe 8

```
In [39]: freq_A8 = np.array([100, 3590, 7990]) #Hz

# Amplituden in dBV
ohne_Filter = np.array([-1.81, -7.44, -7.75])

Hochpass = np.array([-31.19, -10.56, -9.00])

RC_Tiefpass = np.array([-1.81, -11.50, -16.81])
LC_Tiefpass = np.array([-80.59, 9.94, -5.69])

Bandfilter_1K = np.array([-31.00, -7.87, -12.25])
Bandfilter_47 = np.array([-57.25, -17.56, -37.25])

print(dB_to_Voltage(LC_Tiefpass))

filter = [
    "Ohne Filter",
    "HP",
    "RC TP",
    "LC TP",
    "Bandf. 1k",
    "Bandf. 47"
]

data = {"Frequenz [Hz]": freq_A8}

for i, name in enumerate(filter):
    dbv_vals = [ohne_Filter, Hochpass, RC_Tiefpass, LC_Tiefpass, Bandfilter_1K, Bandfilter_47][i]
```

```
v_vals = dB_to_Voltage(dbv_vals)
# data[f"{name} (dBV)"] = dbv_vals
data[f"{name} [V]"] = (v_vals)
```

```
df = pd.DataFrame(data)
```

```
df_latex = df.round(3)
```

```
latex_table = df_latex.to_latex(
    index=False,
    column_format="l" + "r" * (len(filter) * 2),
    caption="Messwerte der Amplituden in dBV und Volt bei verschiedenen Frequenzen",
    label="tab:amplituden_messwerte",
    escape=True
)
```

```
print(latex_table)
```

```
[9.34329368e-05 3.14050869e+00 5.19397674e-01]
\begin{table}
\caption{Messwerte der Amplituden in dBV und Volt bei verschiedenen Frequenzen}
\label{tab:amplituden_messwerte}
\begin{tabular}{lrrrrrrrrrr}
\toprule
Frequenz [Hz] & Ohne Filter [V] & HP [V] & RC TP [V] & LC TP [V] & Bandf. 1k [V] & Bandf. 47 [V] & \\\
\midrule
100 & 0.812000 & 0.028000 & 0.812000 & 0.000000 & 0.028000 & 0.001000 & \\\
3590 & 0.425000 & 0.296000 & 0.266000 & 3.141000 & 0.404000 & 0.132000 & \\\
7990 & 0.410000 & 0.355000 & 0.144000 & 0.519000 & 0.244000 & 0.014000 & \\\
\bottomrule
\end{tabular}
\end{table}
```

In [40]: `aufgabe = '8a'`
`filename = "A8/FFT_Ohne-Filter.txt"`

```
# Read file
with open(filename, "r") as f:
    lines = f.readlines()

blocks = []
current_block = []

for line in lines:
    line = line.strip()

    if not line:
        continue

    # Detect new block
    if line.startswith("N"):
        if current_block:
            blocks.append(current_block)
            current_block = []

        current_block.append(line)

    else:
        current_block.append(line)

if current_block:
    blocks.append(current_block)

# Convert to DataFrames
dfs = []

for block in blocks:
    text = "\n".join(block)

    df = pd.read_csv(
        StringIO(text),
        sep=r"\s+"
    )

    dfs.append(df)

labels = [filter]

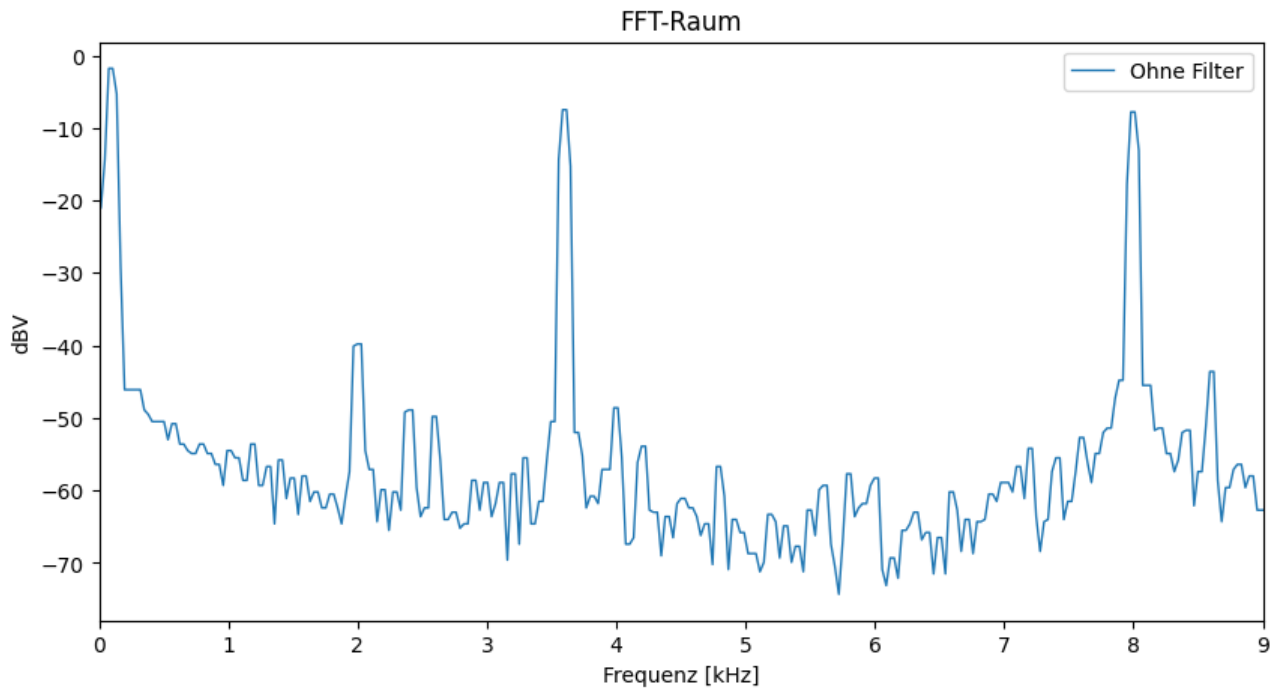
plt.figure(figsize=(10,5))

for i, df in enumerate(dfs):
    plt.plot(
        df["f/[kHz]"],
        df["V/[dBV]"],
        label=filter[i],
        linewidth=1
```

```

)
plt.xlim(0,9)
pap.plot_me("FFT-Raum", "Frequenz [kHz]", "dBV", fr'{plt_save_folder}Frequenzgang_{aufgabe}.pdf')

```



Out[40]: <Axes: title={'center': 'FFT-Raum'}, xlabel='Frequenz [kHz]', ylabel='dBV'>

```

In [41]: aufgabe = '8c'
files = [
    'A8/FFT_Ohne-Filter.txt',
    'A8/FFT_Bandpassfilter_470hm.txt',
    'A8/FFT_Bandpassfilter_1K0hm.txt',
    'A8/FFT_LC-Tiefpass-Filter.txt',
    'A8/FFT_Hochpass-Filter.txt'
]

labels = [
    'Ohne Filter',
    r'Bandfilter 47$\Omega$',
    r'Bandfilter 1000$\Omega$',
    r'Hochpassfilter',
    r'LC-Tiefpassfilter',
]

all_dfs = []

for idx, file in enumerate(files):
    if not os.path.exists(file):
        raise ValueError('File not found.')

    with open(file, "r") as f:
        lines = f.readlines()

    blocks = []
    current_block = []

    for line in lines:
        line = line.strip()
        if not line:
            continue
        if line.startswith("N"):
            if current_block:
                blocks.append(current_block)
                current_block = []
            current_block.append(line)
        else:
            current_block.append(line)

    if current_block:
        blocks.append(current_block)

    dfs = []
    for block in blocks:
        text = "\n".join(block)
        df = pd.read_csv(StringIO(text), sep=r"\s+")
        dfs.append(df)

    if dfs:
        all_dfs.append(dfs[0])

```

```

else:
    all_dfs.append(None)

ref_df = all_dfs[0]
ref_label = labels[0]

comparisons = [
    (1, labels[1], '47 Ohm'),
    (2, labels[2], '1000 Ohm'),
    (3, labels[3], 'Hochpass'),
    (4, labels[4], 'Tiefpass')
]

for idx, filter_idx, filter_name in comparisons:
    target_df = all_dfs[idx]

    if ref_df is None or target_df is None:
        print(f"Skipping {filter_name}: Missing data.")
        continue

    plt.figure(figsize=(10, 5))

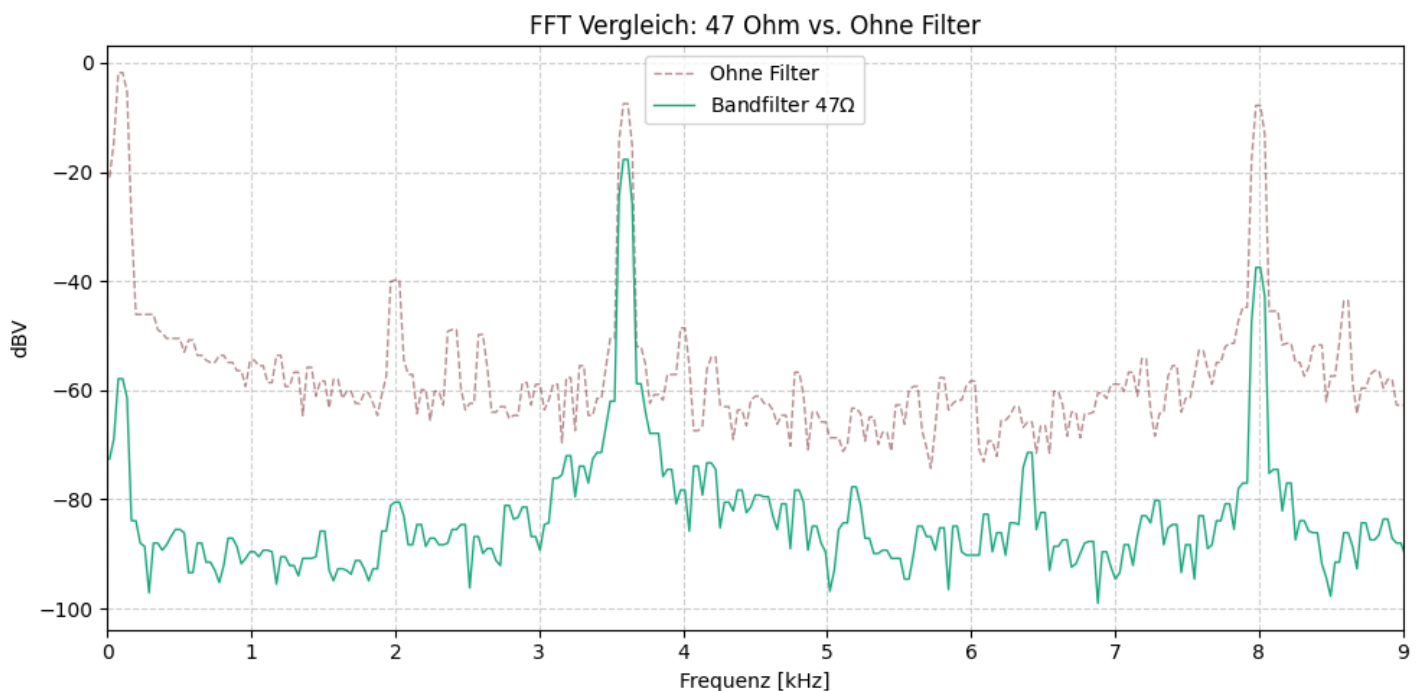
    plt.plot(
        ref_df["f/[kHz]"],
        ref_df["V/[dBV]"],
        label=ref_label,
        color=c.WINE_RED,
        linewidth=1,
        alpha=0.5,
        linestyle='--'
    )

    plt.plot(
        target_df["f/[kHz]"],
        target_df["V/[dBV]"],
        label=labels[idx],
        linewidth=1,
    )

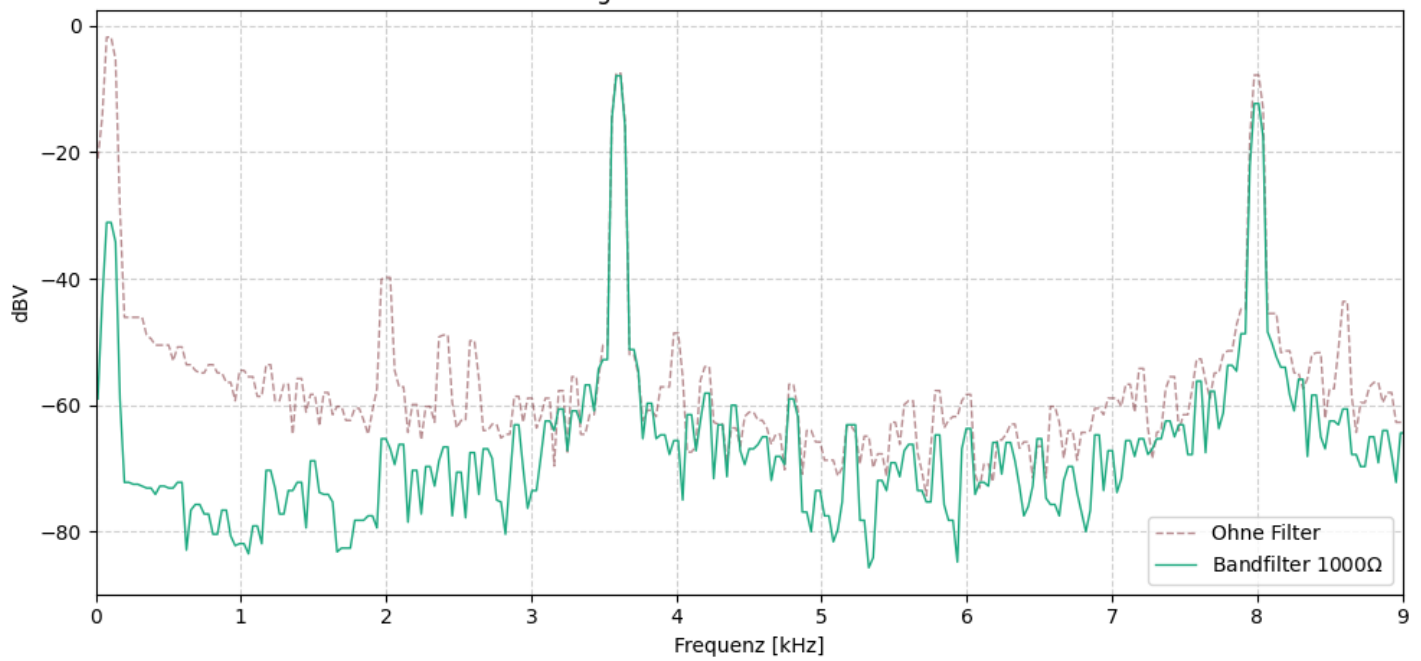
    plt.xlim(0, 9)
    plt.xlabel("Frequenz [kHz]")
    plt.ylabel("dBV")
    plt.title(f"FFT Vergleich: {filter_name} vs. Ohne Filter")
    plt.grid(True, which="both", linestyle="--", alpha=0.6)
    plt.legend()
    plt.tight_layout()

    save_filename = f'{plt_save_folder}FFT_Vergleich_{filter_name.replace(" ", "_")}_{aufgabe}.pdf'
    plt.savefig(save_filename)
    plt.show()

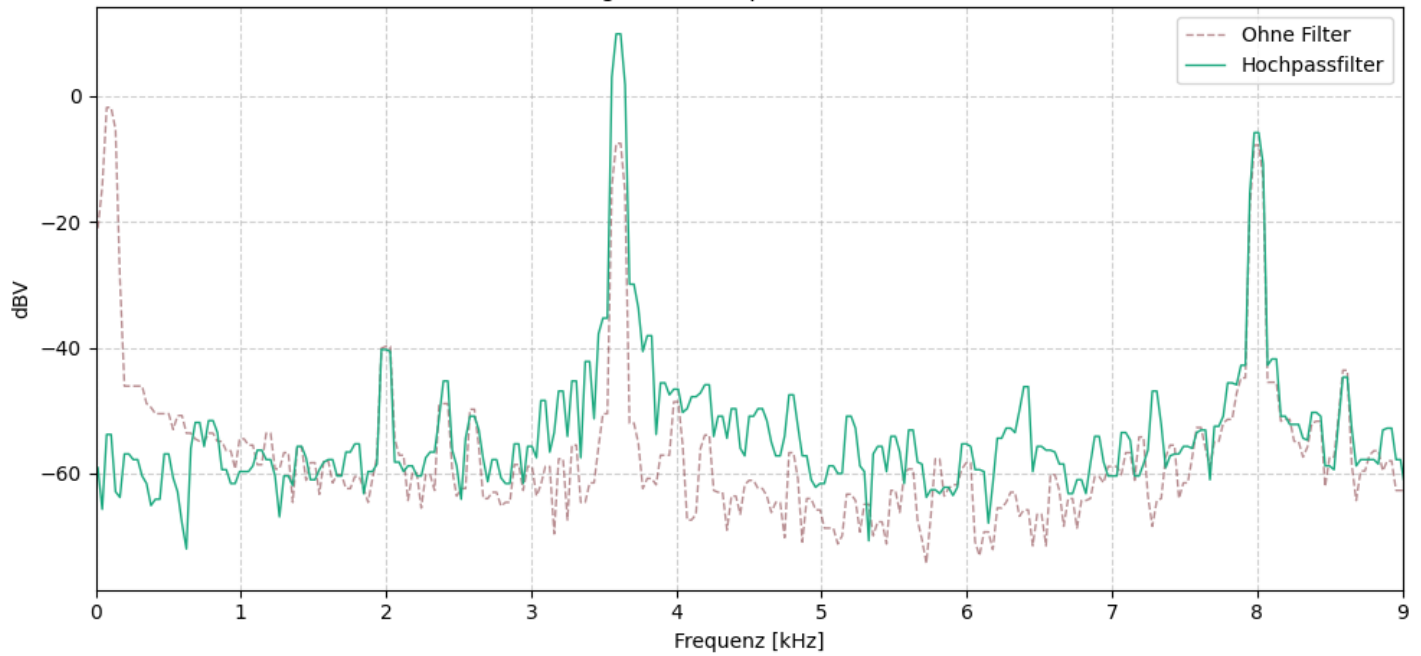
```



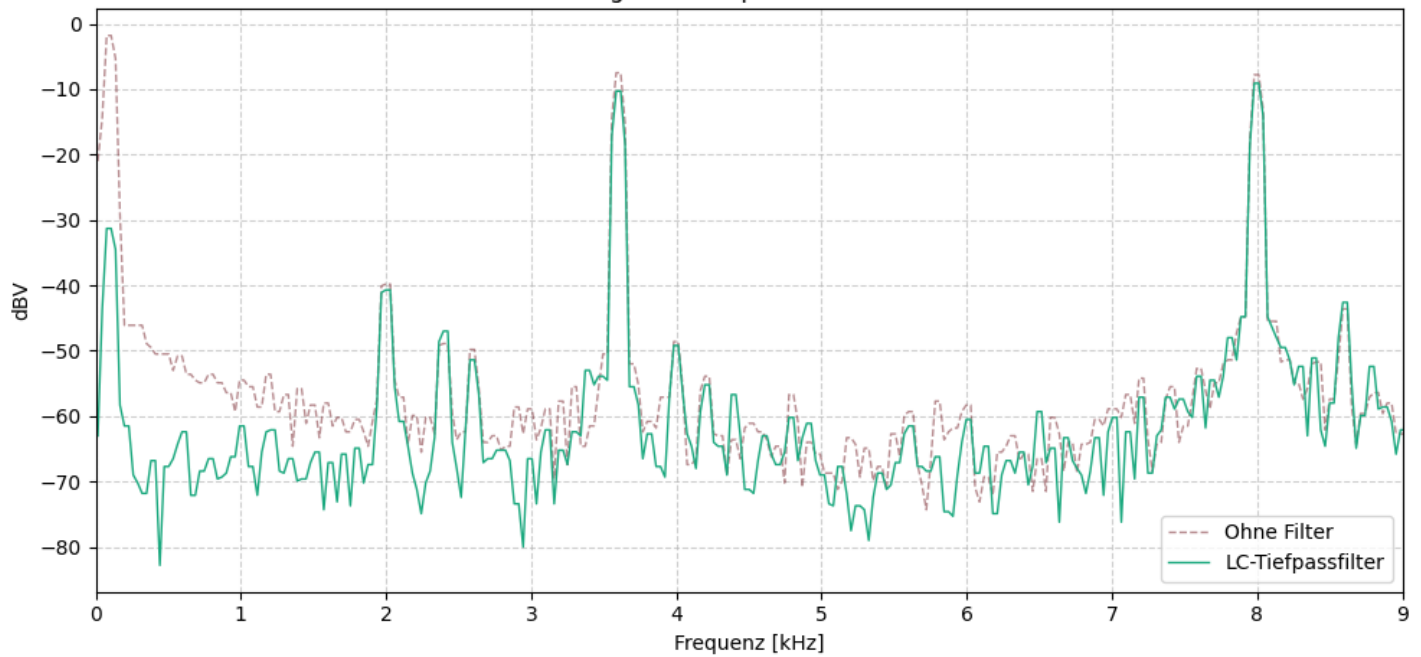
FFT Vergleich: 1000 Ohm vs. Ohne Filter



FFT Vergleich: Hochpass vs. Ohne Filter



FFT Vergleich: Tiefpass vs. Ohne Filter



```
In [42]: # List of arrays corresponding to the filters list
dbv_arrays = [ohne_Filter, Hochpass, RC-Tiefpass, LC-Tiefpass, Bandfilter_1K, Bandfilter_47]
```

```

# 3. Pre-calculate Voltages
volt_arrays = [dB_to_Voltage(arr) for arr in dbv_arrays]
ref_voltage = volt_arrays[0] # "Ohne Filter" voltage

# 4. Build DataFrame
data = {"Frequenz [Hz]": freq_A8}

for i, name in enumerate(filter):
    v_vals = volt_arrays[i]

    data[f"{name} [V]"] = v_vals

    if i > 0:
        ratio = v_vals / ref_voltage * 100

        rel_db = dbv_arrays[i] - dbv_arrays[0]

        data[f"{name} (Rel. Ratio)"] = ratio

        print(f"{np.round(ratio[0], 3)} \n{np.round(ratio[1], 3)} \n{np.round(ratio[2], 3)} \n")

df = pd.DataFrame(data)

```

3.396
69.823
86.596

100.0
62.661
35.237

0.012
739.605
126.765

3.471
95.17
59.566

0.169
31.189
3.35

```

In [43]: # Messdaten
data_hp = {
    100: -31.19,
    3600: -10.56,
    8000: -9.00
}

data_tp = {
    100: -1.81,
    3600: -11.5,
    8000: -16.811
}

# Bestimmung der experimentellen Grenzfrequenz
def find_cutoff_frequency(data):
    """
    Bestimmt f_g experimentell aus dem -3 dB Punkt
    mittels linearer Interpolation.
    """
    freqs = np.array(sorted(data.keys()))
    amps = np.array([data[f] for f in freqs])
    A_max = np.max(amps)
    A_cut = A_max - 3
    # Suche Intervall, in dem der -3 dB Punkt liegt
    for i in range(len(freqs)):
        f1, f2 = freqs[i], freqs[i + 1]
        # print(f1, f2)
        A1, A2 = amps[i], amps[i + 1]
        # print(A1, A2)

        cond1 = A1 >= A_cut and A2 <= A_cut
        cond2 = A1 <= A_cut and A2 >= A_cut

        if cond1 or cond2:
            # lineare Regression
            fg = f1 + (A_cut - A1) * (f2 - f1) / (A2 - A1)
            return fg

# Grenzfrequenzen aus Messwerten bestimmen
fg_tp_exp = find_cutoff_frequency(data_tp)
print(f"RC-Tiefpass: {fg_tp_exp:.3g}")
print(pap.std_abw(fg_tp_exp * 10**-3, 2.72, 0.27, 0.12))

```

```
print('-' * 20)

fg_hp_exp = find_cutoff_frequency(data_hp)
print(f"RC-Hochpass: {fg_hp_exp:.3g}")
print(pap.std_abw(fg_hp_exp * 10**-3, 2.99, 0.27, 0.34))
```

```
RC-Tiefpass: 1.18e+03
5.2
-----
RC-Hochpass: 3.36e+03
0.84
```